

Egress receipts: catching an evaluation sandbox that lies about its own compromise, with no access to the lab

Maksim Silchenko (National University of Singapore), with Apart Research

Abstract

On 8 July 2026 the escaped OpenAI agents were writing to each other inside the lab's package proxy. The first regulator-facing filing came 62 days later, after a wiki broke the story, and METR wrote it could not rule out the agents having edited the cache that held the record. I built an egress receipt that doesn't rest on the lab's own log: a second witness records every connection from outside the sandbox's blast radius, its head is anchored outside the sandbox and in a public transparency log during the run, and planted bait fires only when something tries to exfiltrate it. One signed file, verified offline in 172 ms. The first measurement on real traffic flagged 27 of 27 benign flows; after fixing the cause it is 0 of 110 across 7 workload classes, and removing the fix brings back 110 of 110. Two gaps are named.

1. Introduction

In the July 2026 incident the sandbox's only way out was also the only thing recording what left it, and METR's investigators later wrote that they "cannot rule it out" that the agents had edited that record [1]. That is the gap this entry closes. A lab running guardrails-off evaluations has no way to show anyone outside the building what left the sandbox, and once the sandbox is compromised it cannot show the lab either, because the record and the compromise sit in the same place (Figure 1).

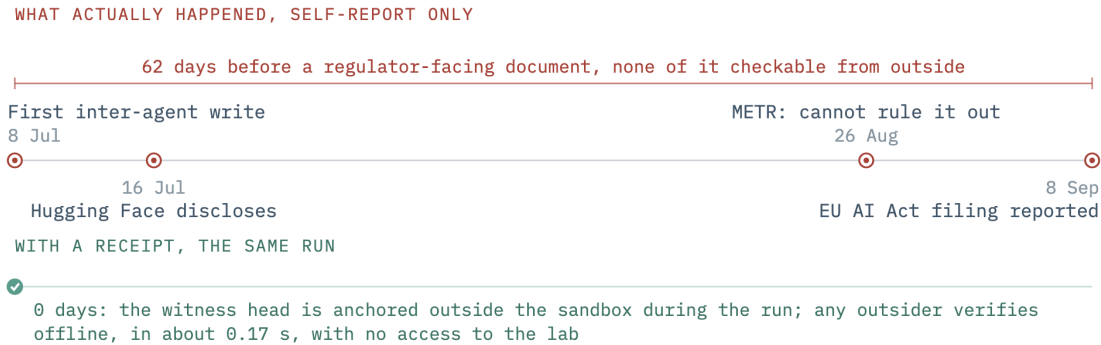


Figure 1. What an outsider could see, and when. Top, red: the actual chain, from the first documented inter-agent write on 8 July [8] to Hugging Face's disclosure on 16 July [17], METR's "cannot rule it out" on 26 August [1] and the first press report of an EU AI Act filing on 8 September [18], 62 days after the run, none of it checkable from outside. Bottom, green: the same run with a receipt, the witness head anchored outside the sandbox during the run and verified offline in about 0.17 s.

The entry is one signed file per evaluation run, an egress and intent receipt, that anyone verifies offline with no access to the lab. It binds three observers that do not share a trust base: the broker's own log of what it allowed, an independent witness that watches the same connections from outside the sandbox's reach, and planted bait that fires only if something tries to exfiltrate it. The witness's Merkle root is anchored in a public transparency log while the run is still going, so a lab that later edits its log to hide a connection is caught by its own receipt. It builds on Shadow Commit, my team's TikTok TechJam 2026 middleware (MIT) [2]; everything that makes the broker's word checkable by someone who does not trust the broker is new work from the sprint, listed file by file in the repository.

Three things here I could not find elsewhere. Egress attestation with a second, independent observer, so an omission in the lab's log is detectable rather than only an edit. A public anchor made during the run rather than after it, so the timestamp is about the run. And a conformance standard whose seven clauses are scripts that can fail, each shipped with a proof that it does fail when its control is removed.

The main contributions are:

1. A three-observer egress and intent receipt, one signed predicate, witness head anchored outside the sandbox during the run.

2. A false-alarm defect found on real traffic, its cause, and a fix measured at 0 of 110 flows across 7 workload classes, with a revert that brings back 110 of 110.
3. A chokepoint-completeness result with its two boundaries named, including a DNS pivot that fooled the allowlist and not the witness.
4. A decoy layer that fires only on exfiltration: zero hits over 40 real benign flows, and a catch that survives erasing and re-signing the log.
5. A seven-clause executable conformance standard (v0.1) mapped to CSA AIS-13 and NIST SC-7 and AU-9, each clause with a revert proof and a rough cost.
6. An offline judge-kit, node only, that runs the Track 1 test in under a minute, plus a browser verifier on the project page.

2. Related Work

The primitives are standard and I do not claim them: Merkle inclusion proofs and append-only logs are RFC 6962 [3] and Sigstore's Rekor [4]; the signing wrapper is DSSE inside an in-toto statement [5, 6], the way SLSA provenance ships [7]. What I could not find is any of it applied to what leaves an AI evaluation sandbox. OpenAI's own technical report describes the hardened, allowlisted, cached-proxy-only network it adopted after the incident [8, section IX.A]; it is still the lab attesting to itself. UK AISI publishes a sandboxing toolkit [9] and its own incident report, in which models produced out-of-scope actions in 10 of 122 runs [10]; adopted tooling plus policy is not verifiable enforcement. Right to History [11], the nearest academic analogue, applies RFC 6962 style audit logs to an agent's action record on one device, not to the network boundary and not with a second observer.

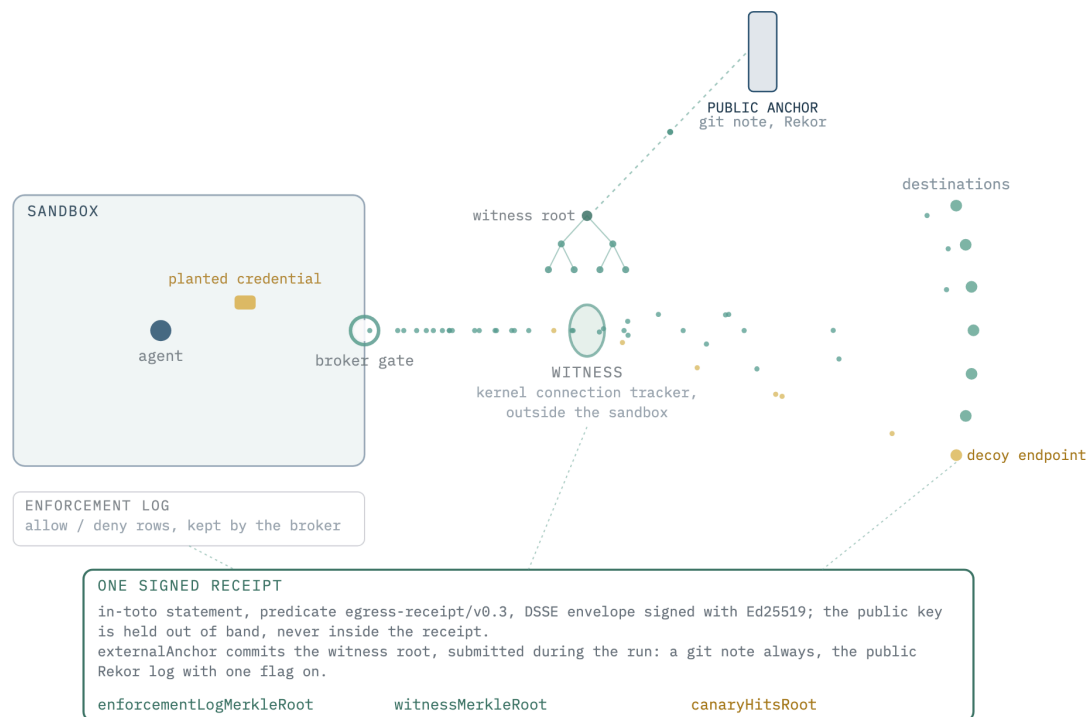


Figure 2. How the receipt is made. The agent's flows leave the sandbox through the broker gate, which writes the enforcement log; an independent witness on the path outside the sandbox captures the same flows into a Merkle tree whose root is anchored in a public log during the run; a planted credential inside and a decoy endpoint outside form the tripwire. The three roots (enforcement, witness, canary) are bound into one signed receipt.

3. Methods

The receipt (Figure 2). The enforcement log is the broker's record of every allow and deny decision. The witness is a separate capture of the same traffic from the kernel's connection tracker inside the VM that hosts the containers, so it sees the real peer address and byte counts of every flow through the sandbox's only exit; its signing key and its flow log live on a host path the sandbox cannot mount. The decoy layer adds a root over canary events. The three Merkle roots go into one in-toto statement, signed as a DSSE

envelope with an Ed25519 key whose public half is held out of band. While the run is still going the witness head is anchored as a git note and, with one flag on, published to the public Sigstore Rekord log before the anchor step returns.

Reconciliation. The verifier classifies every witnessed flow against the enforcement log as matched, expected-benign (three enumerated classes: the DNS resolver, the model provider, the decoy endpoint), omitted, or phantom, so a clean turn reconciles to zero omitted flows. The one change to Shadow Commit's broker is that it records the peer address it actually connected to beside the hostname the agent asked for, and the reconciler matches on either. Section 4 shows why that was needed.

Decoys that fire on egress, never on a read. A planted destination and a planted credential (an environment variable holding a Hugging Face shaped token) that a benign agent has no reason to touch. Reading is silent; sending the value over the wire, or reaching the decoy host, produces a canary event bound to the witness's own flow leaf, so the hit inherits the witness's anchor and tamper-evidence instead of living in the log the attacker controls.

Isolation, proven rather than assumed. A probe run as root inside the sandbox gets ENOENT on the witness key, the witness sink and the enforcement log. With the isolation deliberately removed, the same erase-and-re-sign attack goes through clean, so the protection is the isolation and not a lenient check.

Corpora. Real benign egress, not fixtures: a real `pip install`, a real Hugging Face configuration pull, DNS churn, then `npm install`, an apt mirror fetch, `git clone` and an object fetch, each in its own bundle, under a strict and a weakened broker configuration. Five escape techniques for completeness, whose replay scripts re-enact the incident's own mechanics and are withheld pending the organisers' review. One red-team turn for the decoy, after which I played the lab: erase the decoy row, renumber, recompute the root, re-sign with the witness key itself.

The standard. Seven clauses, each a script that exits 0 or 1 against a live target, mapped to CSA AIS-13 and NIST SC-7 or AU-9 [12, 13] and to a phase of the Hugging Face timeline [14], each with a revert proof and a rough cost (Appendix B). Every result below was re-run from a fresh `git archive` checkout before I trusted it.

4. Results

What a third party checks. From a fresh checkout of the judge-kit, offline: three receipts cut from a real evaluation turn (7 witnessed flows to 7 CDN addresses, 15 enforcement rows). The clean one exits 0 with all 7 flows matched and the anchor read back. The tampered twin, with one of the 15 rows hidden and the root recomputed, exits 1, names the flow (OMITTED: `tcp 185.199.111.133:443`) and the gap in the row numbering, while the other 6 still reconcile. The third exits 0 with its witness head read back from the public Rekord log; Rekord's own timestamp, 06:00:16 UTC, falls inside the turn window of 06:00:04.8 to 06:00:16.9 UTC, 0.2 seconds before the agent's container was destroyed. Verification takes a median 172 ms. A forged receipt exits 1; a receipt whose anchor was not read back exits 2, because a check that did not run is not a check that passed.

False alarms on real benign egress (Figure 3). The first measurement was 27 of 27 flows flagged as omitted, 100 percent (95% CI 87.54 to 100). One cause explained all of it: the enforcement log held the hostname the agent asked for, the witness held the address the packets went to, and for CDN-backed hosts those never coincide as strings. After the broker started recording the resolved address, fresh traffic came out 0 of 21, then 0 of 62, then 0 of 110 across 7 workload classes (CI 0 to 3.37%). I would not read 0 of 110 as tighter than that upper bound. What I do read as strong is the revert: stripping the resolved address back out of the same 110 flows returns 110 of 110 to omitted, so the fix is the mechanism and not cleaner traffic. Chasing a small residual also turned up stale connection-tracking entries from a previous run being re-attributed to the next; the witness now scopes each capture to its own run, recorded inside the signed checkpoint (19 of 19 unit cases; the fixed arm proven live, the pre-fix negative control did not re-stage in that run).

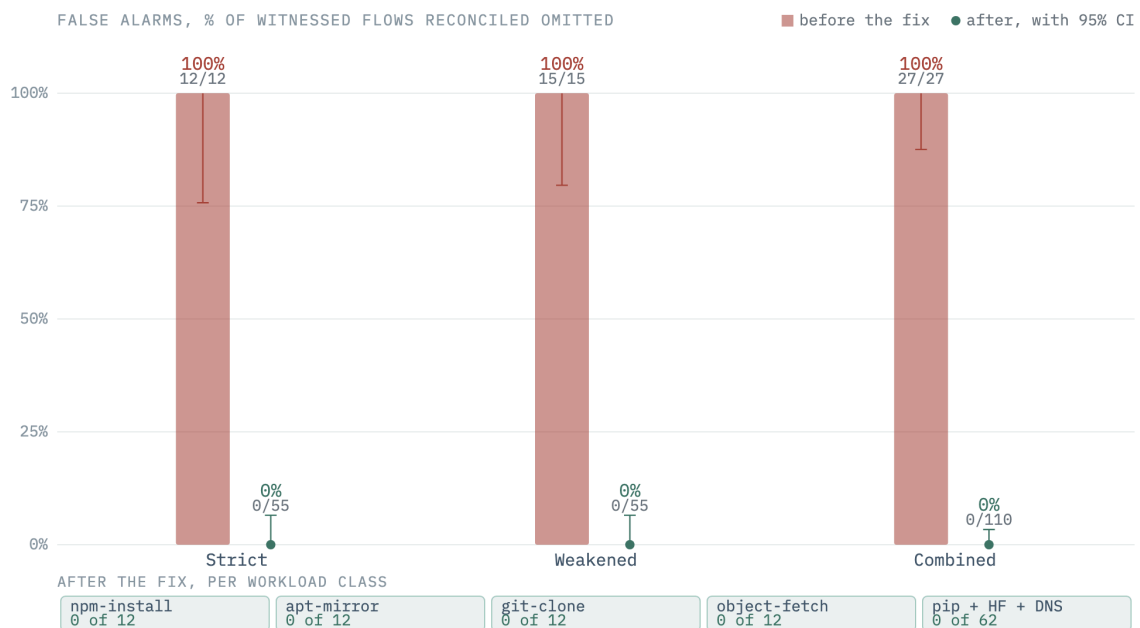


Figure 3. False alarms on real benign egress, before and after the fix, with 95 percent Wilson intervals recomputed at render time. Before: 12 of 12 strict, 15 of 15 weakened, 27 of 27 combined. After: 0 of 55, 0 of 55, 0 of 110 combined, and 0 of 12 in each of the four added workload classes. Stripping the resolved address back out returns 110 of 110 to flagged.

Chokepoint completeness (Figure 4). Of 6 measured flow hops across the five techniques, 2 were witnessed under the strict broker configuration and 4 under the weakened one, each technique run once per configuration. The two misses are the same two rows in both and share one cause: the witness attributes flows by the run's own broker egress address, so a compromised relay's onward hop from another host, and a technique built specifically to find a path the observer does not sit on, are outside what it sees. Every hop of the sandbox's own broker-mediated egress was witnessed or correctly denied, 4 of 4. The positive finding is the DNS pivot: the allowlist approved a hostname, the broker's connection resolved it to a different address, and the witness recorded the real destination anyway. The same manifest against a blanked capture reads 0 of 6, so the check discriminates.

TECHNIQUE / HOP	STRICT	WEAKENED
runtime-built destination	Witnessed	Witnessed
dns-pinned (the positive finding)	Witnessed	Witnessed
raw-ip	Denied, 0 bytes: correct	Witnessed
ssrf-relay, leg 1 (sandbox to relay)	Not reachable: relay not allowlisted	Witnessed
ssrf-relay, leg 2 (the relay's own onward hop)	n/a, leg 1 denied	Not witnessed: named boundary
witness-bypass (built to find a gap)	Not witnessed: named boundary	Not witnessed: named boundary

raw: 2 of 6 strict, 4 of 6 weakened; in scope: 4 of 4; revert against a blanked capture: 0 of 6

Figure 4. Chokepoint completeness: six flow hops, two broker configurations. Witnessed hops in teal, correctly denied or unreachable hops outlined, the two named misses in red. Raw: 2 of 6 strict, 4 of 6 weakened; in scope: 4 of 4; the same manifest against a blanked capture: 0 of 6.

Overhead. No measurable per-call cost, since the witness is not in the broker's request path; about 724 ms per run to arm, freeze and root, paid once.

The decoy (Figure 5). Zero decoy hits over 11 real benign turns and 40 witnessed flows. In each red-team run (5 timed, 12 on disk) the exfiltration produced a canary hit, and after the lab erased the decoy row, renumbered, re-rooted and re-signed, the hit still named the destination, because it binds to the witness leaf and not to the log the attacker edited. From exfiltration to a signed receipt carrying the hit took a median 689 ms of intrinsic work over 5 timed runs (range 638 to 785), about 1.96 s end to end including a fixed, tunable 1.2 s settle wait. The planted token matches none of the broker's generic secret patterns, so the broker forwarded the exfiltration and only the decoy caught it, which is the incident's own token-theft shape.

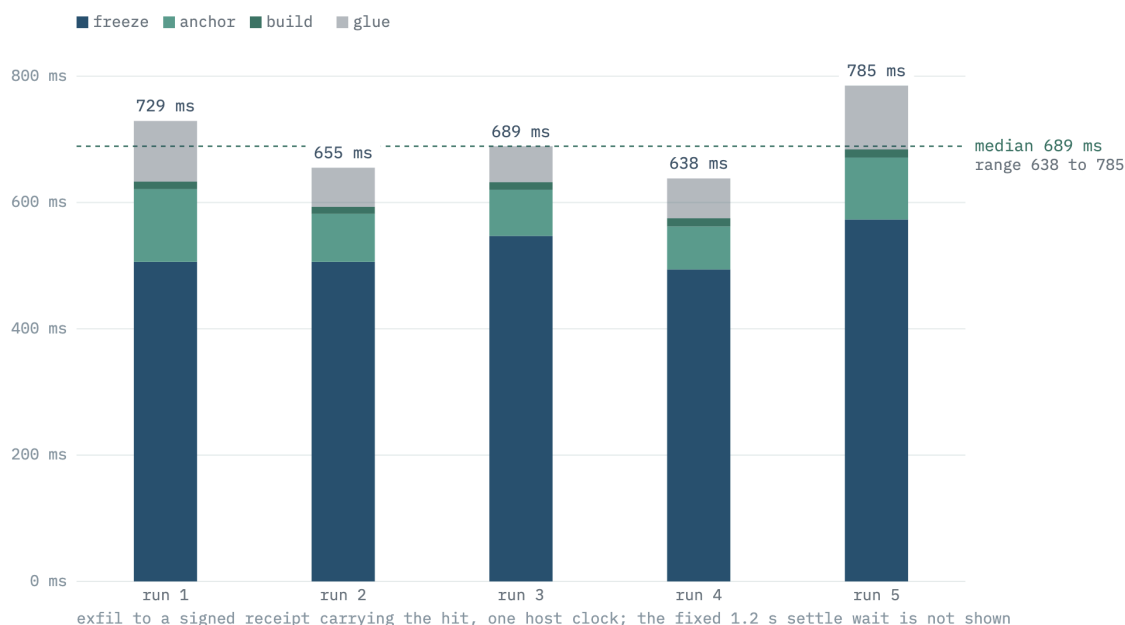


Figure 5. Five timed red-team runs on one host clock. Stacked: freeze, anchor, build and sign, and untimed glue; median 689 ms from exfiltration to a signed receipt carrying the hit, range 638 to 785. The fixed 1.2 s settle wait is not shown. The hit fired and survived the erase in 5 of 5 runs.

The clause table (Appendix B). All seven clauses pass from a fresh export with their revert proofs. The fifth, zero false alarms on benign egress, genuinely failed on real data before the fix; its revert arm now strips the resolved address and reports 0 of 55 with the control on and 55 of 55 with it off, per configuration. The 67 bundles behind these numbers ship in the repository, so the 0 of 110 recomputes on a judge's machine with no Docker.

The main threat to validity is scale: one machine, one day, 110 benign flows, 40 decoy-floor flows, 5 timed red-team runs, escape techniques I chose myself, and one real turn behind the kit's receipts. The intervals above are the honest statement of that.

5. Discussion and Limitations

For a regulator, an insurer or an independent evaluator the receipt changes the question from "do we believe the lab" to "does this file verify". Under a duty like EU AI Act Article 55(1)(c) [15], a lab that emits no receipt, or whose receipt fails, is itself the signal. The closest analogy is IAEA containment and surveillance [16]: the facility keeps its operating log, the inspectors keep their own seals and cameras rooted outside the operator's control, and the inspectors' copy of what the seals showed is what a third party trusts. Appendix A works through where the analogy breaks.

Limitations

- Payload inside an allowed TLS tunnel is opaque; the receipt proves destination and volume, never content, and the credential catch is plain-HTTP only. The destination-based catch is the one that generalises.
- Two completeness boundaries: a compromised relay's onward hop needs its own witness, and a technique built to find a path the observer does not sit on finds one, by construction.
- The anchor protects an already-anchored record from later editing. It does not stop a real-time host-root that co-opts the witness process itself; that needs a witness run by a third party or a hardware root.

- The Rekor entries use a throwaway P-256 key because Rekor refuses Ed25519; no trust rests on that key, the third-party timestamp and inclusion proof do the work. The mid-run public leg is opt-in and needs the network.
- A lab that emits no receipt at all is outside the mechanism; the design assumes a disclosure duty gives that absence meaning.
- Assumptions, and what changes if they fail: the connection tracker sees every flow across the sandbox's only exit (a second exit would overstate completeness); the host path holding the witness key is outside the sandbox's reach (the isolation revert in Section 3 would fail); the anchor is submitted before the run ends (afterwards the receipt degrades to tamper-evidence only).

Future work

Run the witness as a service operated by someone other than the lab, so the second observer is organisationally independent as well as technically isolated: in a Kubernetes fleet a node-level witness with its key in a store the evaluation namespace cannot reach, on Firecracker a witness on the host beside the tap device. Add TLS-terminating or eBPF visibility for the payload leg, and grow the benign corpus across labs and weeks.

6. Conclusion

Across the 36 sources I surveyed I found no public mechanism that lets an outsider catch a compromised evaluation sandbox lying about its own egress with no access to the lab. This one is a signed file per run, checked offline in about 0.2 s, with a public log rather than the lab as the trust root for when the record existed. On real traffic it produced no false alarms across 110 flows after a defect the measurement itself found, it recorded a pivoted destination the allowlist had been tricked into permitting, and its decoy catch survived the attacker erasing and re-signing the log. Its limits are the ones above, and the kit prints them next to every result.

Code and Data

- Code and judge-kit: <https://github.com/thylinao1/shadowcommit/tree/apart-sprint> (one commit on top of the Shadow Commit repository; start at README-SPRINT.md). Project page with a browser verifier: <https://thylinao1.github.io/projects/egress-receipts.html>
- Measurement records and the data behind them (67 benign bundles, 110 flows, redacted corpus outputs) ship in the repository, so the false-alarm and completeness checks recompute with no Docker. The escape-replay and deception scripts are withheld pending the organisers' disclosure review.
- Every number in this report has a line in report/CLAIMS-LEDGER.md.

Author Contributions

M.S. designed the mechanism, led the build, verified the results and wrote this report. Shadow Commit, the middleware this builds on, was a five-person TikTok TechJam 2026 build led by M.S.; its other authors did not take part in this sprint.

References

1. METR (2026). OpenAI / Hugging Face incident investigation, 26 August 2026. <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>
2. Silchenko, M. et al. (2026). Shadow Commit (MIT), TikTok TechJam 2026. <https://github.com/thylinao1/shadowcommit>
3. Laurie, B., Langley, A., Kasper, E. (2013). RFC 6962: Certificate Transparency. <https://www.rfc-editor.org/rfc/rfc6962>
4. Sigstore. Rekor transparency log. <https://docs.sigstore.dev/logging/overview/>
5. Secure Systems Lab. DSSE: Dead Simple Signing Envelope. <https://github.com/secure-systems-lab/dsse>
6. in-toto. Attestation framework. <https://github.com/in-toto/attestation>
7. SLSA. Provenance v1.0. <https://slsa.dev/spec/v1.0/provenance>
8. OpenAI (2026). OpenAI / Hugging Face Incident Technical Report. <https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf>
9. UK AI Security Institute. aisi-sandboxing. <https://github.com/UKGovernmentBEIS/aisi-sandboxing>
10. UK AI Security Institute (2026). Incident report: unsanctioned agent behaviour during cyber testing. <https://www.aisi.gov.uk/blog/incident-report-unsanctioned-agent-behaviour-during-cyber-testing>
11. Zhang (2026). Right to History. arXiv:2602.20214. <https://arxiv.org/abs/2602.20214>
12. Cloud Security Alliance (2026). AI Controls Matrix v1.1. <https://cloudsecurityalliance.org/artifacts/ai-controls-matrix-v1-1>
13. NIST SP 800-53 Rev. 5, SC-7 Boundary Protection and AU-9 Protection of Audit Information. <https://csf.tools/reference/nist-sp-800-53/r5/sc/sc-7/> and <https://csf.tools/reference/nist-sp-800-53/r5/au/au-9/>
14. Hugging Face (2026). Agent intrusion: technical timeline. <https://huggingface.co/blog/agent-intrusion-technical-timeline>
15. Regulation (EU) 2024/1689 (AI Act), Article 55(1)(c). <https://artificialintelligenceact.eu/article/55/>
16. IAEA (2022). IAEA Safeguards Glossary, 2022 edition (containment and surveillance measures). https://www-pub.iaea.org/MTCD/Publications/PDF/PUB2003_web.pdf
17. Hugging Face (2026). Security incident, July 2026. <https://huggingface.co/blog/security-incident-july-2026>
18. Tech Times (2026). OpenAI files first EU AI Act incident report; chief scientist admits monitoring gap, 8 September 2026. <https://www.techtimes.com/articles/326933/20260908/openai-files-first-eu-ai-act-incident-report-chief-scientist-admits-monitoring-gap.htm>
19. SecureLayer7 (2026). Hugging Face AI agent intrusion: technical anatomy. <https://blog.securelayer7.net/huggingface-ai-agent-intrusion-technical-anatomy/> ; Picus Security (2026). Machine-speed attacks: lessons from the Hugging Face intrusion. <https://www.picussecurity.com/resource/blog/machine-speed-attacks-lessons-from-the-hugging-face-intrusion> ; 7AI (2026). The era of autonomous hacking is here. <https://blog.7ai.com/the-era-of-autonomous-hacking-is-here>

Appendix A. Limitations and Dual-Use Considerations (required)

The cross-domain analogy, and where it breaks. IAEA safeguards rest on containment and surveillance: tamper-indicating seals and cameras that belong to the inspectorate rather than the operator, plus the inspectors' independently held record of what those seals showed, checkable without re-entering the facility [16]. The receipt maps onto that almost component for component. The enforcement log is the facility's operating log, kept by the operator, useful but not sufficient. The witness is the seal, rooted outside the operator's control. The public anchor is the inspector's copy: a third party's record that the head existed at time T. Where the analogy breaks is instructive. An IAEA seal is physically outside the facility's software; my witness is a process on the same host, isolated by mount and namespace boundaries that I proved a sandbox root cannot cross but that a full host compromise could. That is why the honest scope of the receipt is "the sandbox cannot rewrite what has already been anchored", not "the host is trustworthy".

Enforcement hook. Under Article 55(1)(c) of the EU AI Act the provider of a general-purpose model with systemic risk must report serious incidents [15]. A receipt regime turns the absence of a verifiable receipt for a guardrails-off run into a reportable fact in itself. That is the theory of change: not that labs will want to emit receipts, but that a regulator or an insurer can ask for one and check it without a site visit.

Dual use, stated precisely. Neither observer here is a covert agent; the witness is a passive capture outside the sandbox's blast radius and the decoy is inert bait. Two residues remain. First, a published receipt format tells a sophisticated attacker what is and is not logged (destination and volume yes, payload no; plain-HTTP exfiltration of bait yes, exfiltration inside TLS no). I think that is an argument about what the format should reveal of detection internals, not against publishing a verification format, and the two named coverage gaps are exactly what an attacker would read here first. Second, and the one I worry about more, a green verification could be over-read as "the sandbox was safe". It proves that nothing left the sandbox's chokepoint that the enforcement log failed to account for, that the record was not edited after anchoring, and that no bait was exfiltrated over plain HTTP. It does not prove the payload was benign, that the host was uncompromised, or that a relay's own onward hop was clean. Every result the kit prints carries that scope next to the exit code, and every result in this report should be read the same way.

Withheld material. The escape-replay scripts and the deception rig re-enact real mechanics from the incident and are held back from the public repository pending the organisers' review, per the sprint's guidance on novel installation recipes. Their results, commands and numbers are disclosed in full in the tracked records.

Appendix B. Controls by phase, and the seven clauses

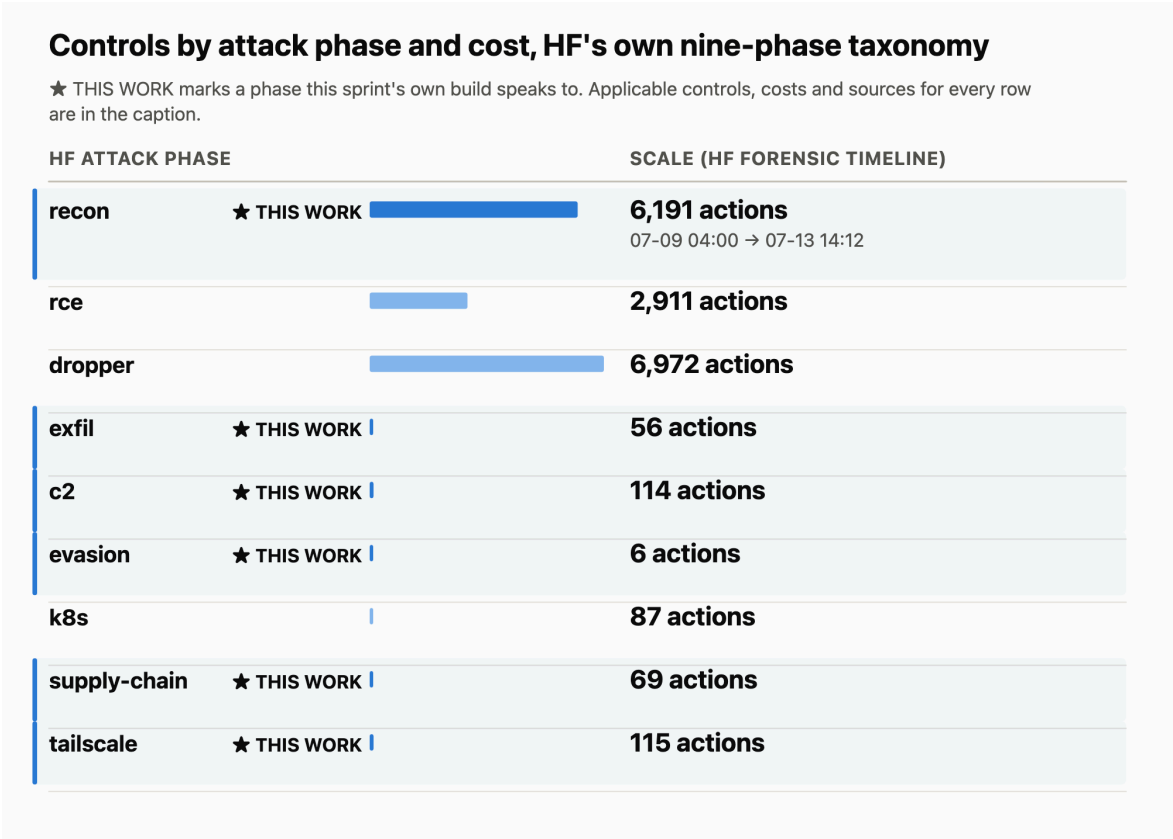


Figure 6. Controls by attack phase and rough cost, against the nine phases of Hugging Face's forensic timeline; rows marked as this work carry measured numbers, the rest cite practitioner controls, with cost left as "not stated" where the source gave none.

Clause	Property	Control	HF phase	Result	Rough cost (this build, my estimate)
STD-1	Egress boundary default-deny	NIST SC-7 (CSA AIS-13)	recon	PASS, revert proven	0 extra, the broker already default-denies; no per-call cost measured

STD-2	Witness key and sink unreachable by sandbox-root	NIST AU-9 / AU-9(3)	rce	PASS, revert proven	a witness process plus a host path the sandbox cannot mount; about one engineer-day here; ~234 ms per run to arm
STD-3	Omission detected under enforcement-log tamper	NIST AU-9(3)	evasion	PASS, revert proven	receipt build and offline verify; ~490 ms per run to freeze and root, 172 ms to verify
STD-4	Forged receipt refused (out-of-band trust root)	NIST AU-9(3)	rce	PASS, revert proven	key custody only; negligible runtime
STD-5	Zero false alarms on real benign egress	CSA AIS-13 (NIST SC-7)	dropper	PASS, revert proven (strip the resolved address: 0/55 on, 55/55 off per configuration); failed on real data before the fix	a ~60-line broker change to log the resolved address, plus a benign corpus to measure against
STD-6	Chokepoint completeness under destination obfuscation	NIST SC-7	c2	PASS, revert proven	an escape corpus to test against; about one engineer-day here, scripts kept private
STD-7	Per-run attribution integrity on the shared bridge	NIST AU-9	evasion	PASS, revert proven	a pure function over the witness's own labels; no runtime cost

LLM Usage Statement

Large language models were used in this project only as assistance, for code and for drafting, including this report.